

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 17

INDEXING: INTRODUCTION

INDEXING: OUTLINE

Indexing Basic Concepts

Ordered Index

B⁺-tree Index

Hash Index



INDEXING BASIC CONCEPTS

1. Assume that you work in a government office, and you maintain the records of **8 million Hong Kong residents**.
2. The record of each resident contains the **hkid**, **address**, **phone number**, etc.
3. People come to your office and ask you to retrieve the records of persons given their hkid, for example:

Find me the record of the person with hkid A634569.

4. Let's forget about computers for now. You just want to keep the records in a **paper-based catalog**, so you answer these queries by manually looking up the catalog.

✎ Assuming you can put **8 records per printed page** (i.e., $bf_{\text{catalog}}=8$), the catalog will be **1 million pages**.



INDEXING BASIC CONCEPTS (CONT'D)

Records: 8,000,000
Records/page: 8
Pages: 1,000,000

How would you arrange the records in the catalog for this query?

- Your goal is to **minimize the cost** (i.e., effort) of finding records.
 - This **cost** is measured **as the number of pages “accessed”** (i.e., the **page access cost**) before finding the record.
-

Strategy 1: ordered randomly

If the catalog records are in **random order** of **hkid**, then in the **worst case** you must **search the entire catalog** (page access cost = 1,000,000) before finding a record, or to determine that the **hkid** does not exist in the catalog.

What would be the average case page access cost?

Strategy 2: ordered on **hkid**

What would be the average case page access cost?



INDEXING BASIC CONCEPTS (CONT'D)

Records: 8,000,000
Records/page: 8
Pages: 1,000,000

- The same considerations apply when we use computers; instead of paper pages, we have **secondary storage pages of a fixed size**.
 - Every time we read something from secondary storage (i.e., do a **page I/O**), we need to **bring an entire page into main memory** (i.e., into the **buffer**).
 - The **major cost** is **how many pages we read** (i.e., **page I/O cost**) because secondary storage page I/O operations are much slower than CPU operations.
- ✎ **Our goal is to minimize the time needed for searching.**
- ✎ **An ordered file reduces the cost to that of binary search.**

Can the cost be reduced further?



INDEXING BASIC CONCEPTS (CONT'D)

Records: 8,000,000
Records/page: 8
Pages: 1,000,000

- Continuing with our catalog example, let's keep the ordered file, but also build an additional **index** (e.g., at the front of the catalog).

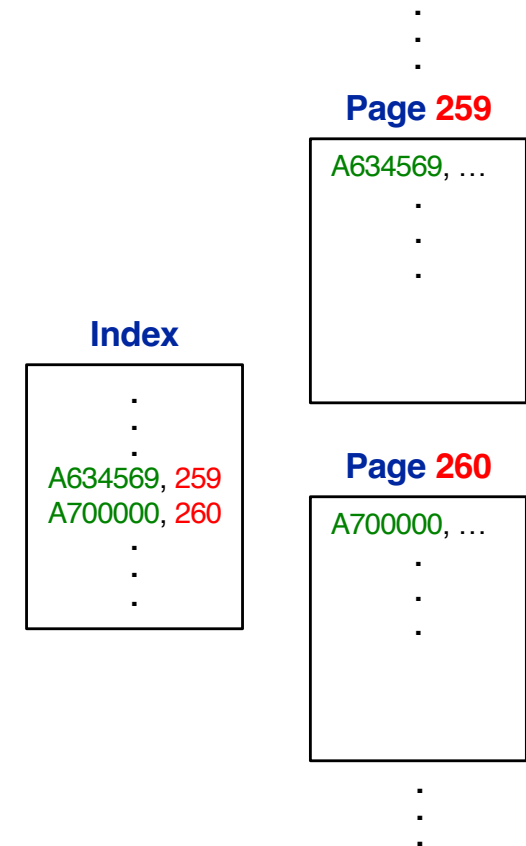
- Each **index entry** is a small record that contains a **hkid** and the catalog page number where you can find this **hkid**.
- For example, $\langle A634569, 259 \rangle$ means that **hkid** **A634569** is on page **259** of the catalog.

👉 **hkid** is the search key of the index.

Recall: A search key is not the same as a primary key or candidate key!

- Each index entry is usually **much smaller** than the actual record.
- Let's assume that **we can fit 100 index entries per paper page** (i.e., $bf_{\text{Index}}=100$).

👉 **The index entries are also ordered on hkid.**





INDEXING BASIC CONCEPTS (CONT'D)

Is an index entry needed for each of the 8,000,000 records?

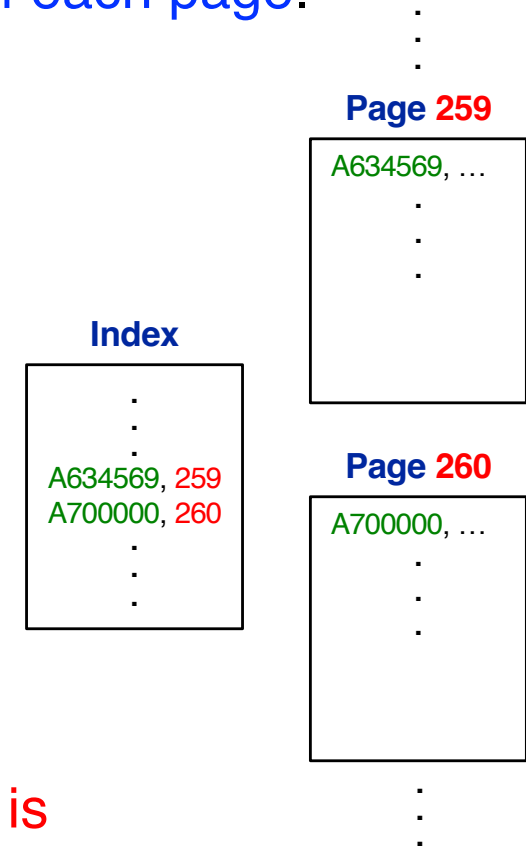
No We only need an entry for the first record of each page.

Example

If there are two consecutive entries $\langle A634569, 259 \rangle$, $\langle A700000, 260 \rangle$ in the index, then we know that every hkid starting from $A634569$ and *up to, but not including*, $A700000$ must be on page 259 .

Therefore, **we need only 1,000,000 index entries** (one for each page of the main catalog).

Since **we can fit 100 index entries per page**, and we have **1,000,000 index entries**, **the index is** $\lceil 1,000,000/100 \rceil = 10,000$ pages (i.e., 10^4 pages).





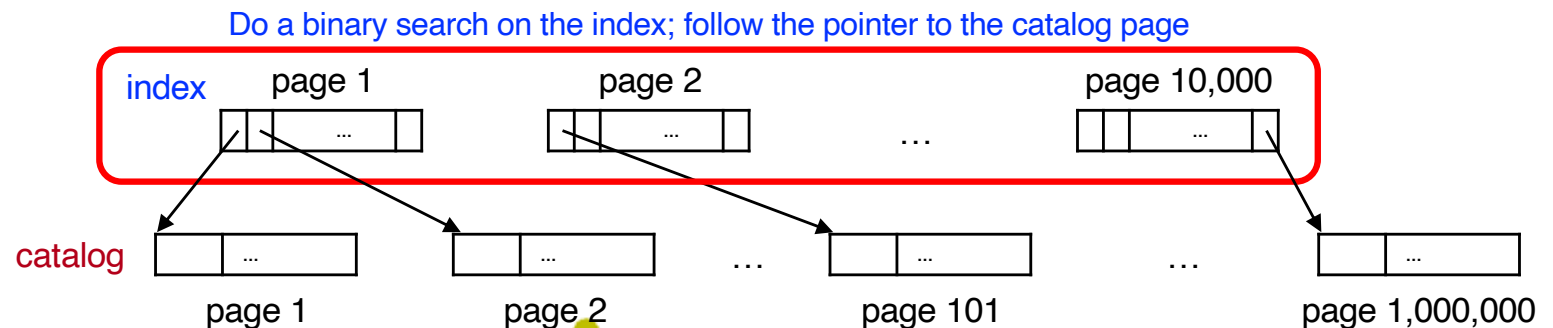
INDEXING BASIC CONCEPTS (CONT'D)

index entries/page: 100
index pages: 10,000 (10^4)

How can the index be used to speed up search for a record?

- Use **binary search on the index** to find the index page containing the largest **hkid** value that is **smaller or equal to** the search **hkid** value.
 - The page access cost for this search is $\lceil \log_2 10^4 \rceil = 14$.
- Then, follow the pointer from that index entry to the actual catalog page.
 - The page access cost for this is 1.

Total page access cost: $14 + 1 = 15$ (Page accesses reduced from 20 $\rightarrow$ 15.)





INDEXING BASIC CONCEPTS (CONT'D)

index entries/page: 100
index pages: 10,000 (10^4)

14.2.2

Can the cost be reduced even further?

Yes Build an **index on the index** (i.e., a second level index)

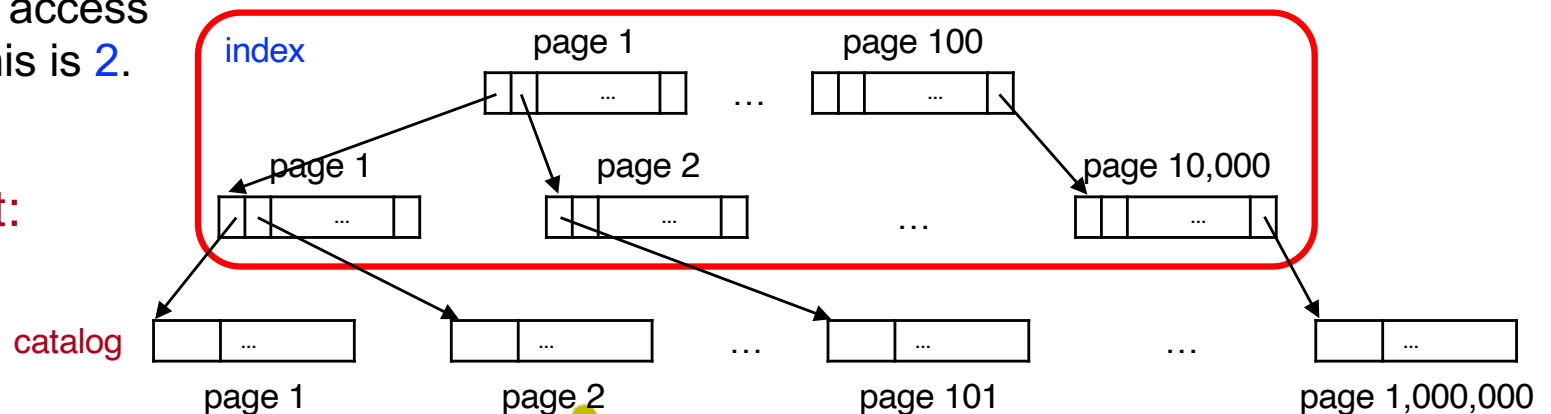
- The **second level index** contains 10,000 index entries, one for each page of the first index, and requires $\lceil 10,000/100 \rceil = 100$ (10^2) pages.
- Use **binary search on the second level index** to find the index page containing the largest **hkid** that is smaller or equal to the search **hkid**.
 - The page access cost is $\lceil \log_2 10^2 \rceil = 7$.
- Then, **follow the pointer** from that index entry **to the first level index** and finally **follow the pointer** in that index **to the actual catalog page**.

➤ The page access cost for this is 2.

Total page access cost:

$$7 + 2 = 9.$$

(Page accesses reduced from 20 → 15 → 9.)





INDEXING BASIC CONCEPTS (CONT'D)

index entries/page: 100
index pages: 10,000 (10^4)

Can the cost be reduced even further?

Yes Build a third level index

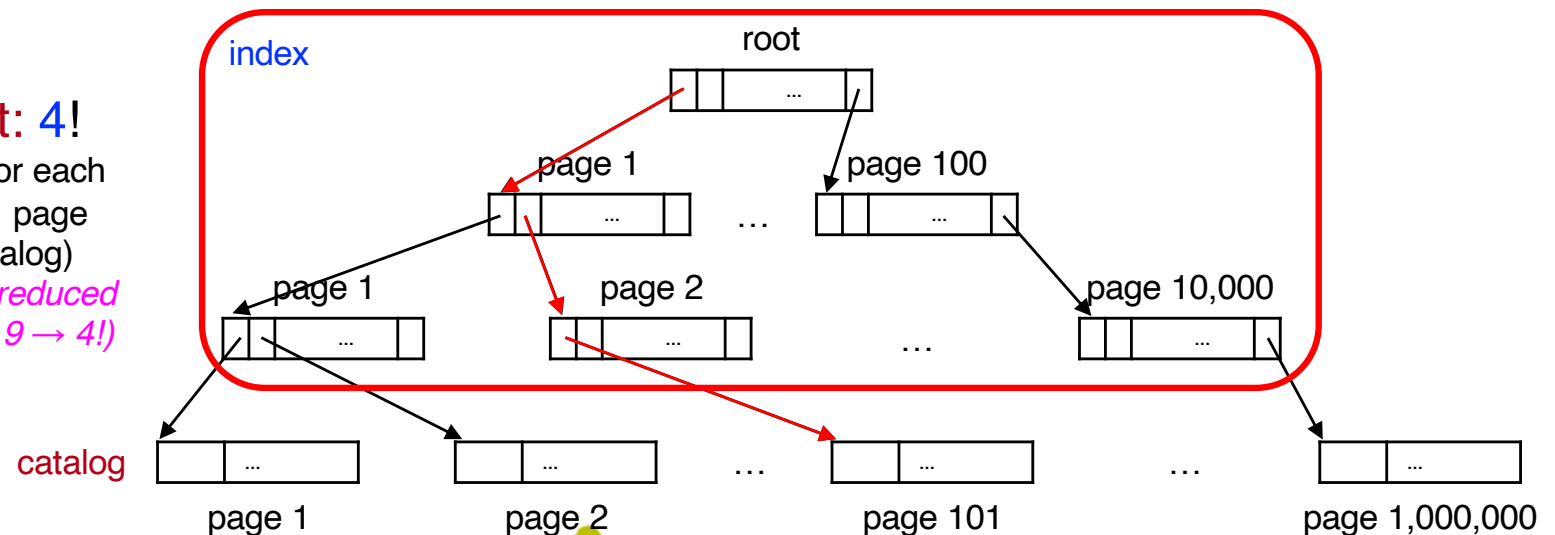
- The **third level index** contains 100 index entries, one for each page of the second level index, and requires $\lceil 100/100 \rceil = 1$ page.
- Search this page to find the largest **bkid** that is smaller or equal to the search **bkid**, and follow the pointer to the second level index, then follow the pointer in the second level index to the first level index and finally follow the pointer in this index to the catalog page.

Total page

access cost: 4!

(1 page access for each index level plus 1 page access to the catalog)

(Page accesses reduced from 20 → 15 → 9 → 4!)





INDEXING BASIC CONCEPTS: RECAP

Search key The attribute, or set of attributes, used to search for records in a file.

👉 **Do not confuse with the concept of primary or candidate key.**

- A primary key is always also a search key.
- A search key is not necessarily a primary key (it can be any attribute).
- In the preceding example, the search key was **hkid** since records were found given a value for **hkid**.

👉 **To find records given a person's name (or another attribute) additional indexes need to be constructed.**

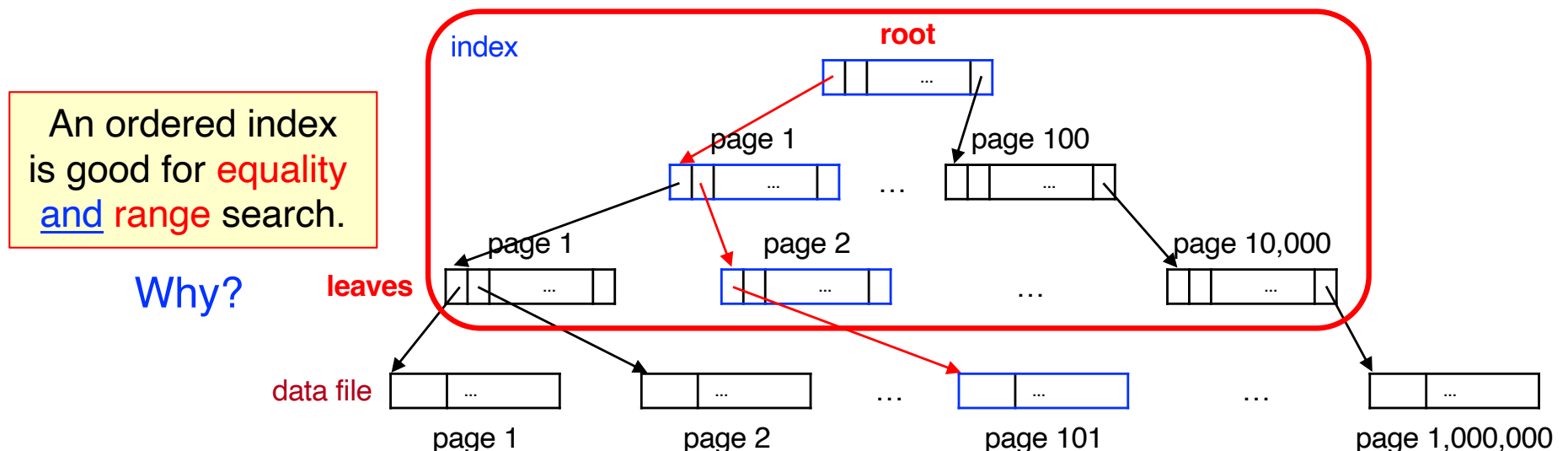
Index file A file consisting of records (called **index entries**) of the form **<search key, pointer>**.

- Index files are typically much smaller than the original file as they do not store all the attributes, but only search-key values and pointers.

ORDERED INDEX

- The index constructed for **hkid** is an **ordered** (or **tree**) index.
 - In each page, the **index entries** are **ordered** (**sorted**) on the search **key** (e.g., **hkid**).
 - **Searching** for a record always **starts from the tree root** and **follows a single path to a tree leaf** using a search key of the record.
 - An **additional access** is then required **to retrieve the record** from the data file.

Page I/O cost: **tree height** (i.e., number of index levels) **plus 1**.



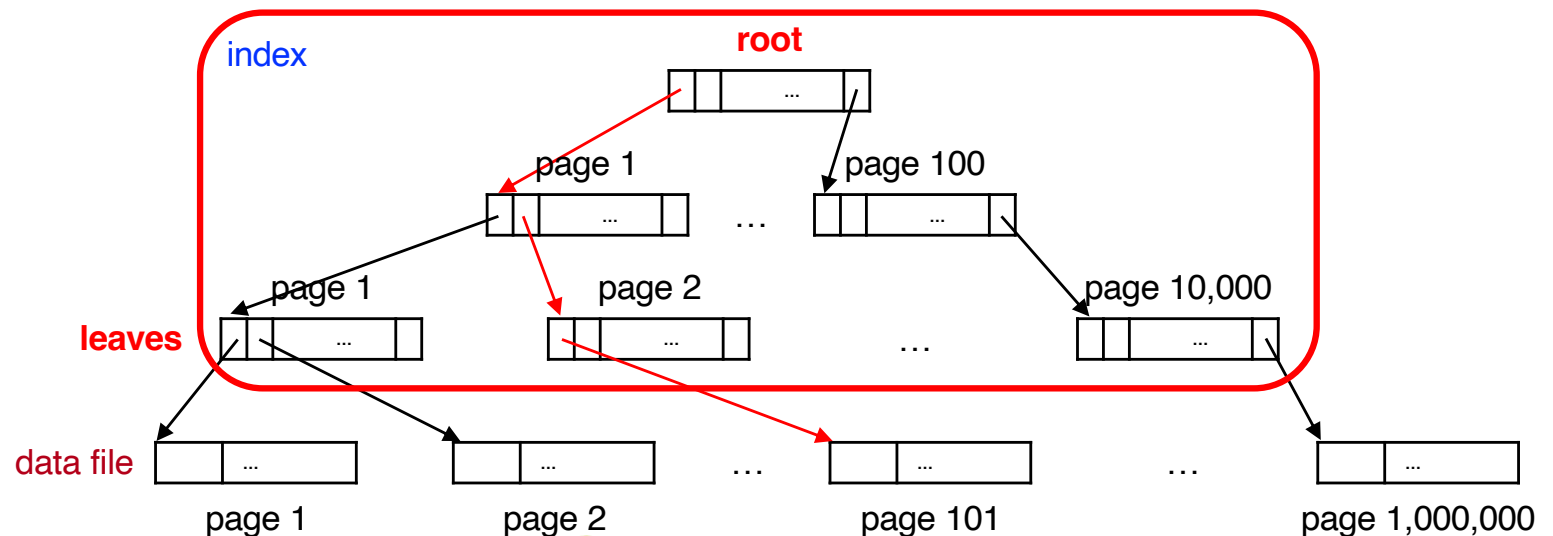
ORDERED INDEX (CONTD)

- An **index page** is also called an **index node**.
- The **number of children** of an index node (i.e., the number of pointers in the node) is called the **fan-out**.

👉 In the catalog example, the fan-out is 100.

- The **tree height** is $\lceil \log_{\text{fan-out}} (\# \text{ of leaf level-index entries}) \rceil$.

👉 In the catalog example, the tree height is $\lceil \log_{100} (10^6) \rceil = 3$.



DENSE VERSUS SPARSE INDEX

Dense Index Contains an index entry in the leaf-level index pages for *every* search-key value in the file.

Sparse Index Contains an index entry in the leaf-level index pages for *only some* search-key values in the file.

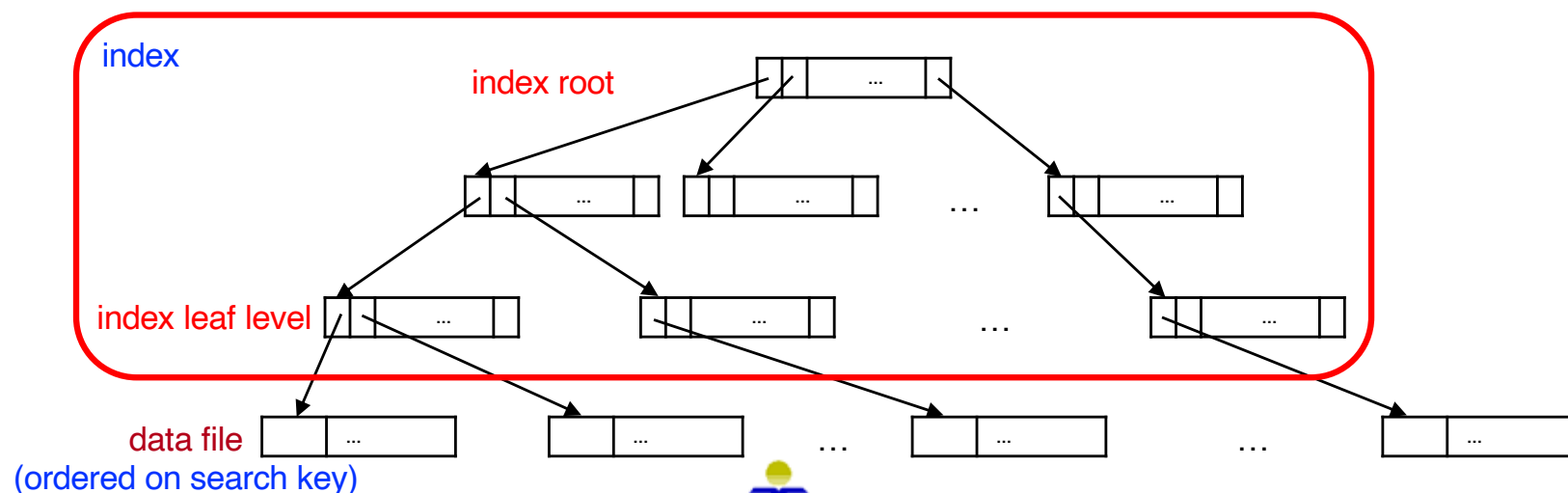
Example: The *hkid* index only has index entries for the first record in each page of the file $\Rightarrow$ sparse index.

- In general, there is an index entry for every data file page corresponding to the minimum search-key value in the page.
- To locate a record with search-key value K (single-level, sparse index):
 - Find the index entry with the largest search-key value $\leq K$.
 - Follow the associated index entry pointer to the data file page.
 - Starting at the first record on this page, search the data file sequentially until the search key value is found or the end of the data file is reached.

👉 **Sparse indexes** require **less space** and **less maintenance overhead** for insertions and deletions than **dense indexes**.

PRIMARY/CLUSTERING INDEX

- A **clustering index** is an index for which the **data file is ordered on the search key** of the index (e.g., the index on **hkid**).
- If the search key of a clustering index is the **primary key**, then the index is called a **primary index**.
 - 👉 **There can only be one primary index for a data file.**
 - 👉 **A primary index is usually sparse. Why?**
- **Index-sequential file**: An ordered, sequential file with a primary index (also called ISAM - indexed sequential access method).

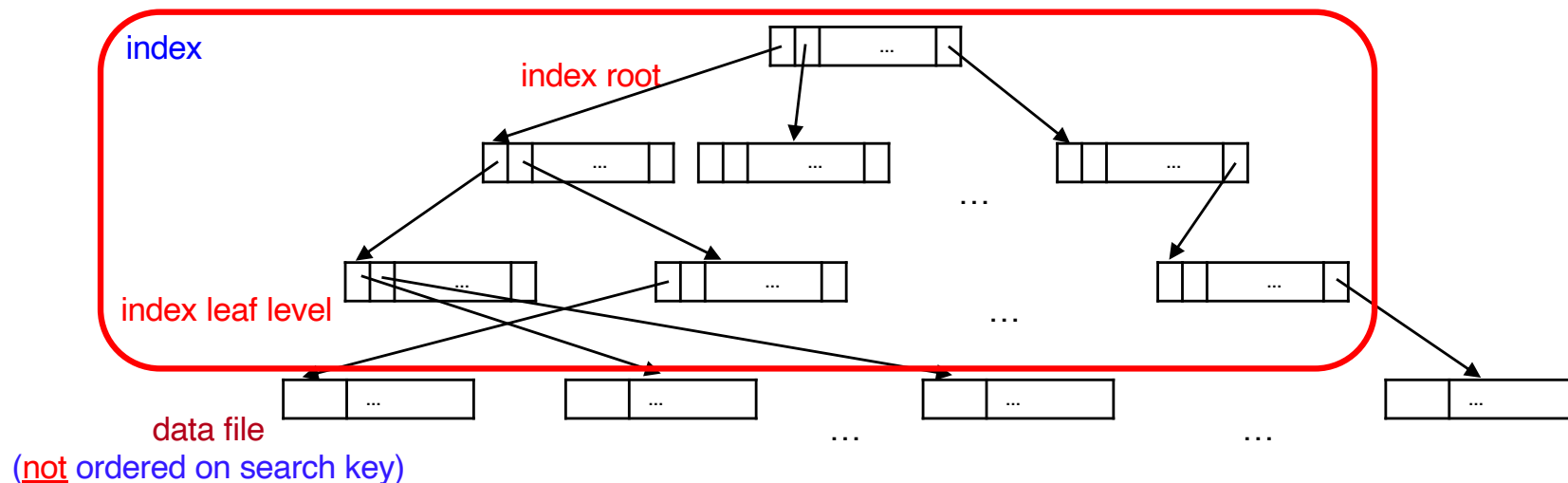


SECONDARY/NON-CLUSTERING INDEX

- A **secondary/non-clustering index** is an index for which the **data file is not ordered** on the search key of the index.

☞ **There can be several secondary indexes for a data file.**

☞ **A secondary index must be dense. Why?**



SECONDARY INDEX EXAMPLE

For the catalog of Hong Kong residents, we also want to be able to find records given a name. **How to find the record fast?**

Solution: Build another index on name

- Since the file is ordered on **hkid**, the new index **must be secondary** (since the file is not ordered on the search key) and **dense** (there is one entry for every search-key value).
- Assuming that all names are distinct (*not realistic!*), the index will contain 8 million entries (one entry for each name).
- Assuming that the fan-out is again 100, the page I/O cost of finding a record given the name is $\lceil \log_{100} 8,000,000 \rceil + 1 = 4 + 1 = 5$.

index height

Recall
Primary index
page I/O cost is 4.

- A **secondary index** is **almost as good as a primary index** (in terms of cost) **when retrieving a single record**.
 - However, it may be very expensive when retrieving many records (e.g., for range queries) and it requires additional storage space because it must be dense.



INDEX ON NON-CANDIDATE SEARCH KEY

We want to build an index on name, but there may be several people with the same name.

⇒ Zero, one or more records are retrieved.

👉 **Not a problem if the index is clustering and sparse.**

How would you do it?

INDEX ON NON-CANDIDATE SEARCH KEY (CONT'D)

If the index is secondary and dense.

Strategy 1: Use variable length index entries

- Each entry contains a name and pointers to all records with this name.

Example: $\langle \text{Jackie Chan}, \text{pointer}_1, \text{pointer}_2, \dots, \text{pointer}_n \rangle$

Problem: Complicated implementation as a file organization that supports records of variable length is needed.

Strategy 2: Use multiple index entries per name

- There is an entry for every person even if he/she shares the same name with other people.

Example: $\langle \text{Jackie Chan}, \text{pointer}_1 \rangle, \langle \text{Jackie Chan}, \text{pointer}_2 \rangle, \dots, \langle \text{Jackie Chan}, \text{pointer}_n \rangle$

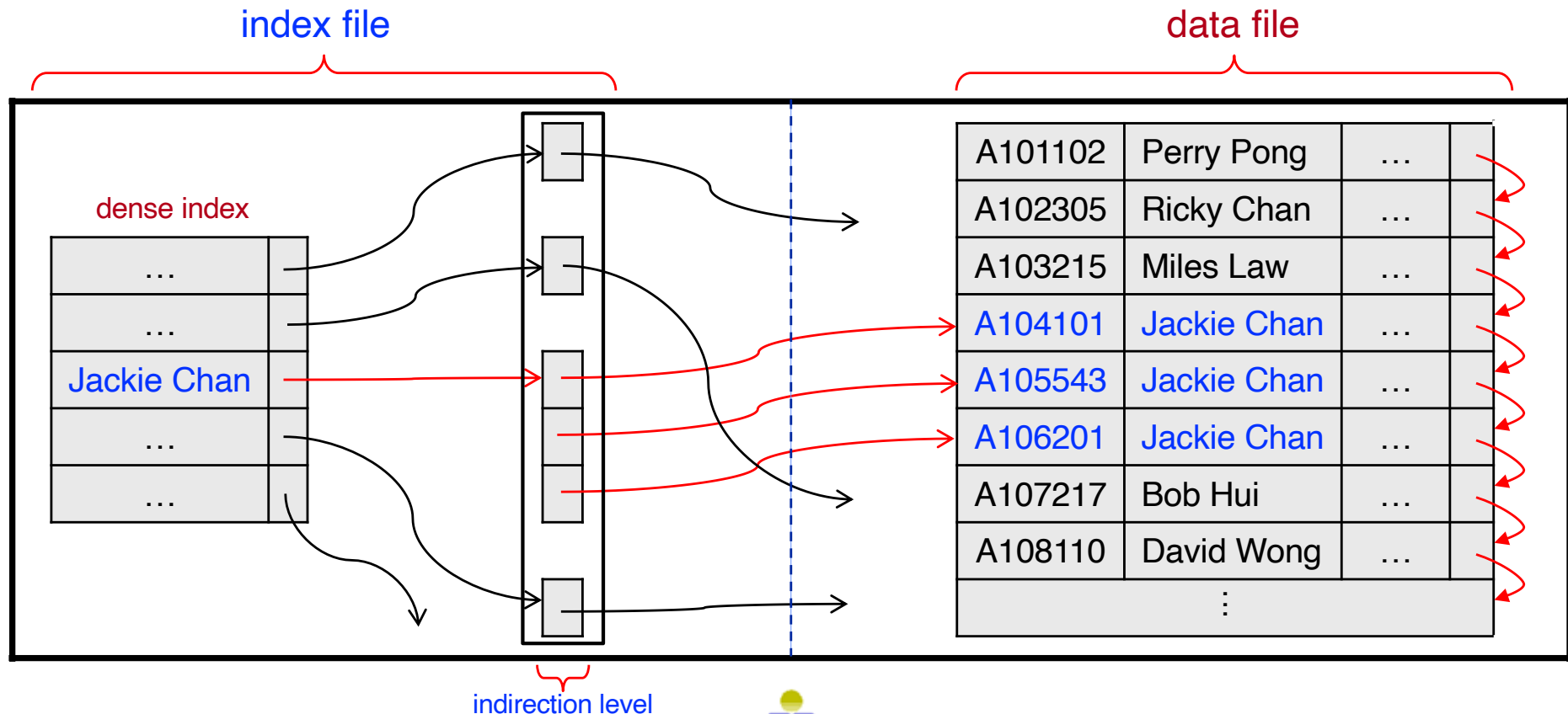
Problem: Redundancy – the name repeats many times.

INDEX ON NON-CANDIDATE SEARCH KEY (CONT'D)

Strategy 3: Use an extra level of indirection (most common approach)

- An index entry points to a page that contains the pointers to all the records with the same name $\Rightarrow$ usually requires at most one additional page access.

 Also referred to as an **inverted file** or **postings list/file**.



INDEX ON COMPOSITE SEARCH KEY

- If a query often uses certain **combinations of attributes** together (e.g., **hkid** and **age**), then creating an **index on this attribute combination** can speed up retrieval.

👉 **A composite search key is a search key that consists of more than one attribute.**

- The index structure for a composite search key is the same as that for a single attribute search key.
- For a composite search key, the ordering of search key values is the **lexicographic ordering**.

Example: For two search keys (a_1, a_2) and (b_1, b_2) :

$(a_1, a_2) < (b_1, b_2)$ if **either** $a_1 < b_1$ **or** $a_1 = b_1$ and $a_2 < b_2$

- This is basically the same as **alphabetic ordering of words**.